

Experiment - 1

Addition

Aim:- To write an R program to perform addition of two numbers.

Program:-

num1 $\leftarrow$ 10

num2 $\leftarrow$ 20

x $\leftarrow$ num1

y $\leftarrow$ num2

z $\leftarrow$ x + y

Print(z)

output

Print(z)

= 30

Subtraction - Experiment - 2

Aim:- To write an R program to perform subtraction of two numbers.

Program:-

num1 $\leftarrow$ 10

num2 $\leftarrow$ 20

x $\leftarrow$ num1

y $\leftarrow$ num2

z $\leftarrow$ x - y

Print(z)

output

Print(z)

= 10

Experiment - 3

Multiplication

Aim:- To write an R program to perform Multiplication of two numbers.

Program:-

```
num1 ← 10  
num2 ← 20  
x ← num1  
y ← num2  
z ← x * y  
print(z)
```

Output

print(z)
= 200

Experiment - 4

Division

Aim:- To write an R program to perform Division of two numbers.

Program:-

```
num1 ← 10  
num2 ← 20  
x ← num1  
y ← num2  
z ← x / y  
print(z)
```

Output

print(z)
= 2

Experiment-5

odd or even

Aim:- To write an R program to check whether a number is odd or even.

Program:-

```
num ← 10
```

```
x ← num
```

```
if (x % 2 == 0) {  
  print("even")  
} else {  
  print("odd")  
}
```

Output:-

even.

Experiment-6

Mean, Median, Mode

Aim:- To write an R program to find the Mean, Median, Mode of a dataset.

Program:-

```
x ← c(10, 20, 30, 40, 50)
```

```
print(mean(x))
```

```
print(median(x))
```

```
print(mode(x))
```

Output

Mean = 30

median = 30

Mode = numeric.

Experiment - 7

Summary

Aim:- To write an R program to display the summary of a dataset.

Program:-

```
x <- c(10, 20, 30, 40, 50)
```

```
summary(x)
```

Output:-

Min	1st Q	Median
10	20	30

Mean	3rd Q	Max
30	40	50

Experiment - 8

Greatest Among three numbers

Aim:- To write an R program to find the greatest among three numbers.

Program:-

```
a <- 10
```

```
b <- 20
```

```
c <- 30
```

```
x <- a
```

```
y <- b
```

```
z <- c
```

```
if (x > y && x > z) {
```

```
  print(x)
```

```
} else if (y > z) {
```

```
  print(y)
```

```
} else {
```

```
  print(z)
```

Output

```
print(z)
```

```
30
```

Experiment-9

IQR

Aim:- To write an R program to find the interquartile Range (IQR)

Program:-

```
x ← c(10,20,30,40,50)
```

```
Print (IQR (x))
```

Output:-

```
Print(IQR(x))=20
```

x

Experiment-10

Quantile

Aim:- To write an R program to find the quantiles of a dataset.

Program:-

```
x ← c (10,20,30,40,50)
```

```
print (quantile (x))
```

output

0%.	25%.	50%.	75%.	100%.
10	20	30	40	50

Experiment - 11

Mid Range

Aim:- To write an R program to find the Mid Range of a dataset.

Program:-

$x \leftarrow c(10, 20, 30, 40, 50)$

$a \leftarrow \max(x)$

$b \leftarrow \min(x)$

$z \leftarrow (a+b)/2$

$\text{print}(z)$

Output:

$\text{print}(z) = 30$

Experiment - 12

z-Score Normalization

Aim:- To write an R program for z-score Normalization.

Program:-

$x \leftarrow c(10, 20, 30, 40, 50)$

$a \leftarrow \text{mean}(x)$

$b \leftarrow \text{sd}(x)$

$z \leftarrow (x-a)/b$

$\text{print}(z)$

Output

-1.2649111 -0.6324555

0.0000000 0.6324555

1.2649111

Experiment-13

Min, Max, Mean and Min-Max Normalization

Aim:- To write an R-program to find Maximum, Minimum, Mean, Min-Max

Program:-

$x \leftarrow c(10, 20, 30, 40, 50)$

$a \leftarrow \min(x)$

$b \leftarrow \max(x)$

$c \leftarrow \text{mean}(x)$

$z \leftarrow (x-a)/(b-a)$

print(a)

print(b)

print(c)

print(z)

output

0.00 0.25 0.50 0.75

1.00

Experiment-14

Barplot and Horizontal Barplot

Aim:- To write an R program to draw a Barplot and Horizontal Barplot.

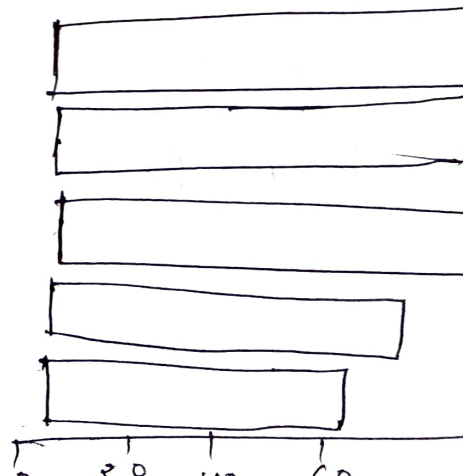
Program:-

$a \leftarrow c(55, 67, 89, 80, 90)$

barplot(a)

barplot(a, horiz=TRUE)

output



Experiment-15

Box plot

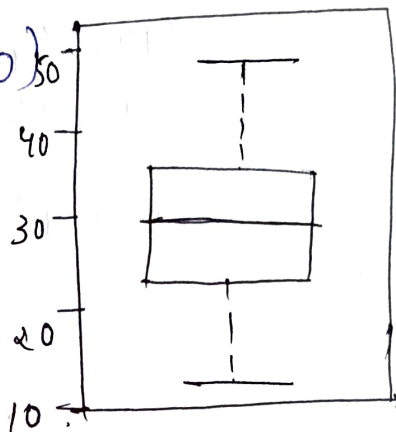
Aim:- To write an R program to draw a Box plot.

program:-

```
x ← c(10, 20, 30, 40, 50)
```

```
boxplot(x)
```

output



Experiment-16

Histogram

Aim:- To write an R program to draw a Histogram.

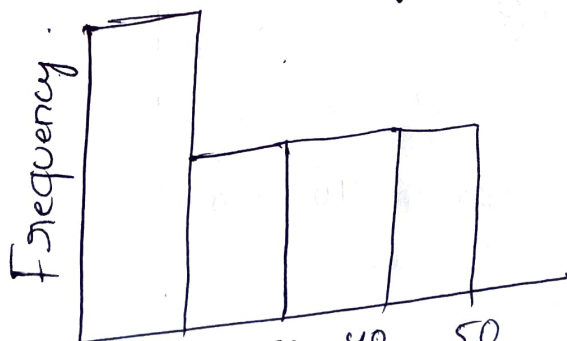
program:-

```
x ← c(10, 20, 30, 40, 50)
```

```
hist(x)
```

output:-

Histogram of x



Correlation Experiment -17

Aim:- To write an R program to perform Correlation Analysis.

Program:-

```
x <- c(10, 20, 30, 40, 50)
```

```
y <- c(15, 25, 35, 45, 55)
```

```
z <- cor(x, y)
```

```
print(z)
```

output:-

print(z) = 1

Scatterplot Experiment -18

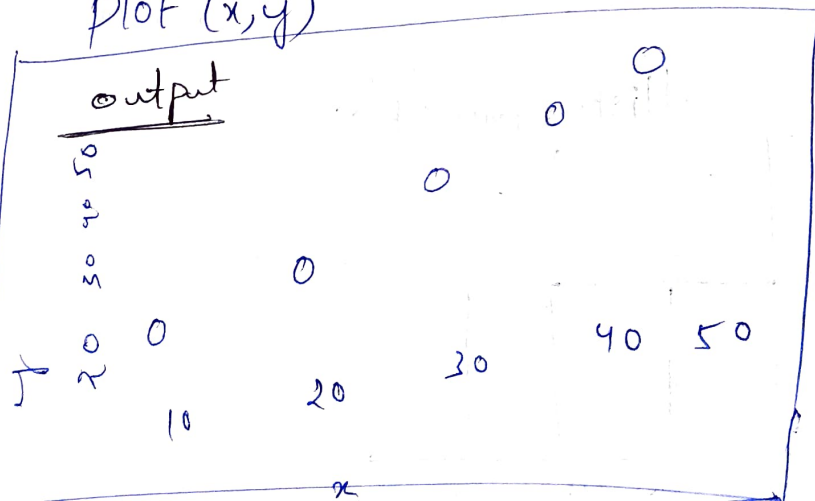
Aim:- To write an R program to draw a scatter plot.

Program:-

```
x <- c(10, 20, 30, 40, 50)
```

```
y <- c(15, 25, 35, 45, 55)
```

```
plot(x, y)
```



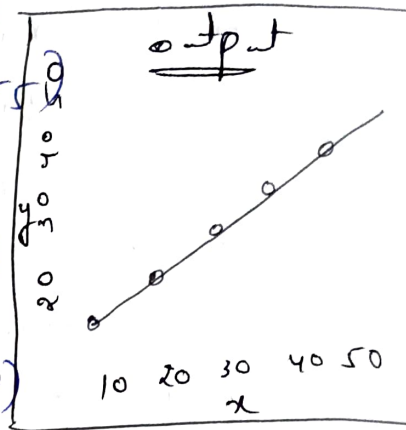
Experiment-19

Linear Regression

Aim:- To write an R program to perform Linear Regression.

Program:-

```
x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 35, 45, 55)
model <- lm(y ~ x)
print(summary(model))
plot(x, y)
abline(model, col = "red")
```



Experiment-20

Multiple Regression

Aim:- To write an R program to perform Multiple Regression.

Program:-

```
data <- data.frame(
  Age = c(20, 25, 30, 35, 40)
  BP = c(120, 125, 130, 135, 140)
  Glucose = c(80, 90, 100, 110, 120)
```

output
call
lm(formula = Age ~ BP + Glucose, data = data)

```
x <- data
model <- lm(Age ~ BP + Glucose, data = x)
print(summary(model))
```

Apriori Algorithm.

Aim:- To find frequent itemsets using Apriori algorithm.

Algorithm:-

1. Input transaction dataset.
2. Generate L_1 (frequent 1-itemsets).
3. Generate L_2
4. Generate L_3
5. Stop when no more itemsets.
6. Generate rules.

Notepad code:-

```
@ relation apriori
@ attribute Rice {yes, no}
@ attribute sugar {yes, no}
@ attribute milk {yes, no}
@ attribute Tea {yes, no}
@ attribute Bread {yes, no}
@ attribute Butter {yes, no}
@ data
```

yes, yes, yes, yes, no, no, no

yes, no, yes, yes, yes, no, no

no, no, yes, no, yes, yes, no

yes, yes, no, yes, yes, no.

Output:-

1. Bread = no $\Rightarrow$ Rice = yes
2. Rice = yes $\Rightarrow$ Bread = no
3. Rice = yes $\Rightarrow$ Butter = no
4. Sugar = yes $\Rightarrow$ Butter = no
5. Bread = no $\Rightarrow$ Butter = no

Result:-

Frequent pattern
Exist among
Rice, sugar, Milk.

Experiment-22

Aim:- To generate frequent itemsets using Apriori

Algorithm:-

1. Input dataset
2. Apply Apriori
3. Apply FP-Growth
4. Compare results.

Notepad code:-

@ relation apriori 2

@ attribute B {yes,no}

@ attribute R {yes,no}

@ attribute E {yes,no}

@ attribute A {yes,no}

@ attribute D {yes,no}

@ attribute M {yes,no}

@ attribute U {yes,no}

@ attribute T {yes,no}

@ data

Output:- Apriori

1. $I = no \Rightarrow B = yes$

2. $I = no \Rightarrow B = yes$

3. $K = no \Rightarrow B = yes$

10. $I = no \Rightarrow B = yes$

FP-Growth

1. $(C=no): B=yes$

$(K=no):$

$(K=no): \Rightarrow (L=no):$

$(I=no): \Rightarrow (I=no):$

$(I=no): \Rightarrow (C=no, K=no)$

Result:-

FP growth gives same result but faster.

Aim:- (Experiment-23)
TO classify data using Decision Tree
and Logistic Regression.

Algorithm:-

1. Load dataset
2. Preprocess data
3. Apply J48
4. Apply logistic
5. Compute accuracy.

Notepad code:-

@ relation bank
@ attribute age numeric
@ attribute balance numeric
@ attribute loan {yes, no}
@ attribute default {yes, no}

@ data

25, 1000, no, no
30, 1500, no, no
35, 2000, yes, yes
40, 2500, no, no
45, 3000, yes, yes
50, 3500, no, no
55, 4000, yes, yes
60, 4500, no, no

Output:-

Decision Tree (J48)

Correctly classified Instance = 5 (62.5%)

Incorrectly classified Inst = 3

Kappa statistic = 0

Total Instance = 8

Result :- Logistic Regression perform

Logistic Regression

= 8 (100%)

= 0

= 1

= .8

Experiment-27

Aim:- To generate frequent itemsets and rules from book dataset (Books ARFF + Association Rule).

Algorithm:-

1. Create ARFF
2. Apply Apriori
3. Apply FP-growth
4. Compare.

Notepad code:-

@relation books
@ attribute Datamining {yes,no}
@ attribute DBMS {yes,no}
@ attribute AI {yes,no}
@ attribute OS {yes,no}
@data

yes	yes	yes	no
no	yes	no	yes
yes	no	yes	no
no	yes	yes	no
yes	yes	no	yes
no	no	yes	yes.

Output Apriori : $L_1 = 8, L_2 = 13, L_3 = 3$

Rules:

OS $\Rightarrow$ AI
AI $\Rightarrow$ DBMS
DBMS $\Rightarrow$ AI

Result:- Apriori and FP-growth produce similar rule.

Experiment 25

Aim:- To compare Decision Tree and Rule-based classification.

Algorithm:-

1. load dataset
2. Apply J48
3. Apply JRIP / PART
4. Compare accuracy.

Notepad code:-

- @ relation credit.
- @ attributes age numeric
- @ attributes income numeric
- @ attributes student {yes, no}
- @ attributes rating {fair, excellent}
- @ attributes class {good, bad}
- @ data.

25, 1000, no, fair, good
30, 4000, no, excellent, good
35, 5000, yes, fair, bad
40, 6000, no, fair, good
45, 7000, yes, excellent, bad
50, 8000, no, excellent, good
55, 9000, yes, fair, bad
60, 10000, no, fair, good.

Output:-

Decision Tree (J48) Rule-based (JRIP)

Accuracy = 62.5% 100%

Correctly clas = 5 8

Incorrect d = 3 0

Result:-

JRIP classifier performs better than J48, PART.

Experiment - 21

Aim:- TO create ARFF dataset and compare
Naive Bayes and decision Tree
Using accuracy & F1-score

Algorithm steps:-

1. convert dataset into ARFF format
2. load dataset in weka
3. Apply Naive Bayes & J48 classifier
4. compare accuracy & F1-score
5. Identify best classifier

Output:-

Decision Tree (J48)

correctly classified Instance = 8

Accuracy = 57.14%

Incorrectly classified Instance = 6

F1 - Measures = 0.508

Confusion Matrix:-

a b

5 4 1 a = yes

3 2 1 b = no

Decision Tree:-

Outlook

= sunny

= overcast = gain.

Windy

humidity

yes (40)

= strong

= high

= normal

= weak

yes (30)

no (20)

Naive Bayes output

Correctly classified instance = 7

Accuracy = 50%

Incorrect classified instance = 7

F_1 - Measures = 0.539

Confusion matrix

a b

7 2 | a = 9

4 1 | b = 5

Result:- Decision Tree classifier
Performed better than Naive
Bayes because it achieves
higher accuracy and F1-score

Experiment - 27

Aim:- To perform k-means clustering and Visualize employee data in Weka

Algorithm steps.

1. create csv dataset
2. load dataset into Weka
3. Apply simple kmeans clustering
4. set number of clusters.
5. visualize clusters.

Notepad code:-

Employee ID, Gender, Age, salary, credit

111, Male, 28, 150000, 39

222, Male, 25, 150000, 27

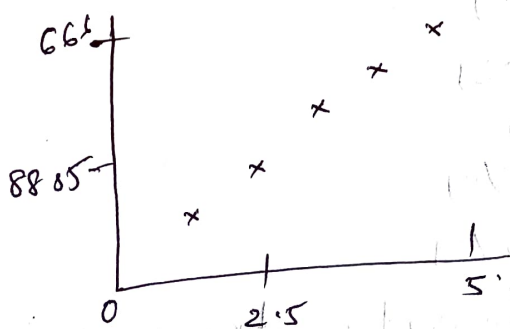
333, Female, 26, 160000, 42

444, Female, 25, 160000, 40

555, Female, 30, 120000, 64

666, Male, 29, 20000, 72

Output:- cluster visualize.



Result:- Employees are grouped successfully based on Salary and credit score.

Experiment - 28

Aim:- To predict diabetes using Decision Tree and Support Vector Machine classifiers in Weka

Algorithm Steps:-

1. Load diabetes dataset
2. Apply J48 Decision Tree.
3. Apply SMO (SVM)
4. Compare Accuracy and F1 score

Notepad code:-

@ relation diabetes

@ attribute glucose numeric

@ attribute bmi numeric

@ attribute age numeric

@ attribute diabetes {yes, no}

@ data

120, 25, 30, no

140, 30, 35, yes

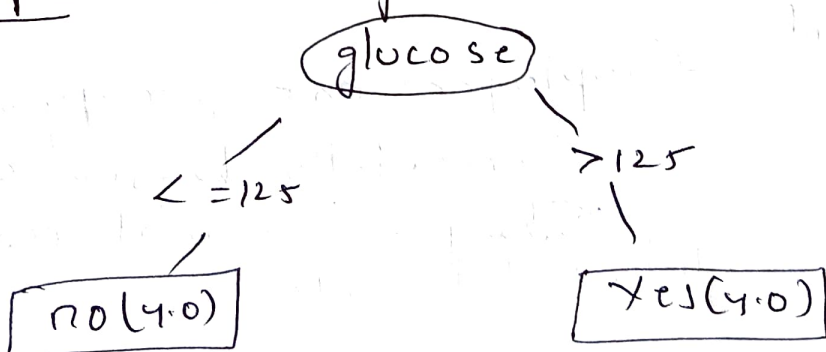
130, 28, 40, yes

110, 22, 25, no

150, 35, 50, yes

160, 38, 55, yes

Output:- Classify - trees - J48 (Decision Tree)



Result:- SVM classifier performs better

Experiment-29

Aim:- To identify the most Significant attribute using info gain evaluator.

Algorithm steps:-

1. create ARFF dataset
2. load dataset
3. open select Attributes tab
4. choose info gain Attribute Eval
5. Apply Ranker Search method.
6. Rank attributes.

Notepad code:-

@ relation studentgrades

@ attribute Marks numeric

@ attribute Attendance numeric

@ attribute Result {pass, fail}

@ data

85, 90, Pass

78, 85, Pass

92, 45, Pass

35, 40, Fail

88, 92, Pass

30, 35, Fail

25, 30, Fail

Output:- Ranked attributes

0.971 2 attendance

0.921 1 Marks

Selected attributes : 2, 1, 2

Result:- Marks attribute has the better information gain.

Experiment-30

Aim:- To perform k-means clustering and Visualize clusters in Weka

Algorithm steps:-

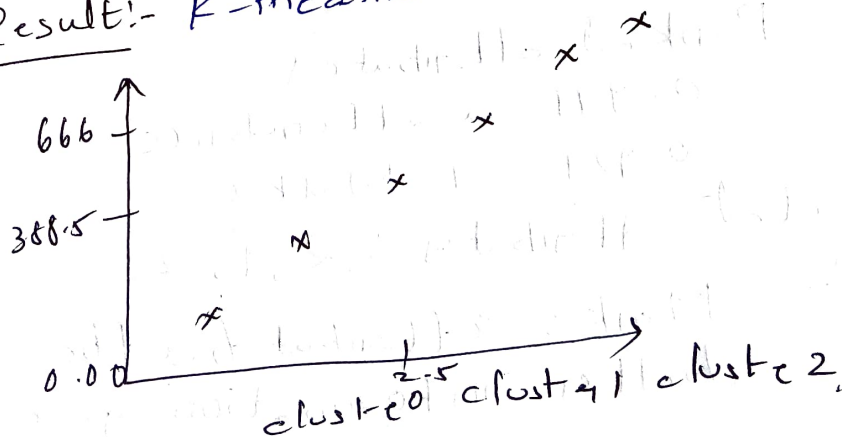
1. Create csv dataset
2. Load dataset into weka
3. Apply simple Means
4. Set cluster size
5. Visualize output.

Notepad code:-

employee ID, Gender, Age, Salary, Credit
111, Male, 28, 150000, 39
222, Male, 25, 150000, 27
333, Female, 26, 160000, 42
444, Female, 25, 160000, 40
555, Female, 30, 170000, 64
660, Male, 29, 200000, 72

Output:- 3 clusters formed
→ High Salary and Low Salary
groups identified.

Result:- k-means Successfully



Experiment-31

Aim:- To generate rules from a Decision Tree and Compare with Rule-Based induction.

Algorithm steps.

1. Create ARFF dataset
2. Load into Weka
3. Apply J48
4. Apply JRip/PART
5. Compare accuracy and confusion matrix.

Notepad code

@relation gender

@attribute height numeric

@attribute weight numeric

@attribute gender {male, female}

@data

185, 85, male

175, 70, male

158, 50, female.

170, 60, female

182, 90, male

160, 55, female

190, 100, male

165, 58, Female

output:- classify → J48 → start (visualize tree)

height

≤ 170

Female (11.0)

> 170

male (4.0)

Result: JRip generated accurate rule and performed better than J48